

Éléments de programmation 2 - LU1IN002

Examen du 12 mai 2026

1h30

Consignes :

- Aucun document n'est autorisé.
- Les calculatrices, baladeurs et autres appareils électroniques sont interdits. Les téléphones mobiles doivent être éteints et rangés dans les sacs.
- Le barème sur 56 points (12 questions) n'a qu'une valeur indicative. Les questions sont indépendantes.
- Il ne sera pas nécessaire de tester la valeur de retour de la fonction `malloc`.

Dans ce sujet d'examen nous allons nous intéresser à l'analyse fréquentielle de relevés d'observations dans une réserve naturelle. Des biologistes parcourent une réserve et notent les espèces animales observées. Chaque espèce est identifiée par un code entier compris entre 1 et `NB_ESPECES` inclus. La constante `NB_ESPECES` est définie au début du programme :

```
#define NB_ESPECES 8
```

Un relevé d'observations est un tableau d'entiers dans lequel chaque case contient le code de l'espèce observée. Par exemple, le relevé suivant indique que la première observation est une espèce de code 3, la deuxième de code 1, etc. :

3 1 5 3 1 3 7 5 1 3

L'objectif est de construire un registre des espèces observées sous forme de liste chaînée dans laquelle chaque espèce n'apparaît qu'une seule fois avec son nombre d'observations. Ce registre sera trié par code d'espèce croissant.

Toutes les fonctions sont à écrire en langage C.

1 Tableaux

Question 1 (4 points)

Écrire la fonction de prototype `int EstValide(int code)` qui retourne 1 si le code d'espèce `code` est valide, c'est-à-dire compris entre 1 et `NB_ESPECES` inclus, et 0 sinon.

Écrire ensuite la fonction de prototype `int CompterOccurrences(int *releve, int n, int code)` qui retourne le nombre de fois où le code d'espèce `code` apparaît dans le tableau `releve` de `n` éléments.

Attention : La simplicité et la clarté de vos fonctions seront prises en compte dans la notation.

Question 2 (4 points)

Écrire la fonction de prototype `void AfficherReleve(int *releve, int n)` qui permet d'afficher dans le terminal les `n` codes d'espèces contenus dans le tableau `releve`. L'affichage des codes se fera sur une seule ligne, séparés par des espaces, avec un retour à la ligne à la fin.

Question 3 (4 points)

Écrire la fonction de prototype `int *ReleveAleatoire(int n)` qui alloue un tableau de `n` entiers, le remplit avec des codes d'espèces valides tirés aléatoirement (chaque code entre 1 et `NB_ESPECES` ayant la même probabilité d'être tiré) puis retourne son adresse. Pour tirer aléatoirement les codes vous

utilisez la fonction `rand()` de la bibliothèque standard du C et dont le prototype est `int rand()` et qui retourne un entier positif compris entre 0 et `RAND_MAX`.

Question 4 (6 points)

On souhaite analyser un relevé en calculant le nombre d'espèces distinctes observées et en identifiant l'espèce la plus fréquemment observée.

Par exemple, pour le relevé 3 1 5 3 1 3 7 5 1 3 de 10 observations, il y a 4 espèces distinctes (les espèces 1, 3, 5 et 7) et l'espèce la plus fréquente est l'espèce 3 (observée 4 fois).

Écrire la fonction de prototype `int AnalyserReleve(int *releve, int n, int *pNbDistinct, int *pCodeMax)` qui retourne le nombre total d'observations (`n`) et qui, en utilisant les pointeurs `pNbDistinct` et `pCodeMax`, fournit respectivement le nombre d'espèces distinctes observées et le code de l'espèce la plus fréquemment observée. En cas d'égalité, on conservera le plus petit code.

Indication : Vous pourrez utiliser la fonction `CompterOccurrences` de la question 1.

2 Listes

Le registre des espèces sera stocké dans une liste simplement chaînée triée par code d'espèce croissant. Voici la structure pour un élément de cette liste :

```
1 typedef struct _tyEspece tyEspece;
2 struct _tyEspece{
3     int code;           /* code de l'espece */
4     int nbObs;          /* nombre d'observations */
5     tyEspece *suiv;     /* pointeur vers le suivant */
6 };
```

Le relevé 3 1 5 3 1 3 7 5 1 3 produira le registre suivant (trié par code croissant) :

(code:1, nbObs:3) → (code:3, nbObs:4) → (code:5, nbObs:2) → (code:7, nbObs:1) → NULL

Question 5 (6 points)

Écrire la fonction de prototype `tyEspece *InsererTrie(tyEspece *registre, int code)` qui permet d'ajouter une observation d'espèce de code `code` dans le registre trié par code croissant. Si l'espèce est déjà présente dans la liste, son champ `nbObs` est incrémenté de 1. Sinon, un nouvel élément est alloué, initialisé avec `nbObs` à 1 et inséré à la bonne position pour conserver l'ordre croissant des codes. La fonction retourne un pointeur vers la tête de liste.

Question 6 (4 points)

Écrire la fonction de prototype `int NbEspèces(tyEspece *registre)` qui retourne le nombre d'espèces (c'est-à-dire le nombre d'éléments) présentes dans le registre `registre`.

Question 7 (4 points)

Écrire la fonction de prototype `tyEspece *DetruireRegistre(tyEspece *registre)` qui permet de libérer toute la mémoire allouée pour la liste `registre` et qui retourne NULL.

Question 8 (6 points)

Écrire la fonction de prototype `void AfficherRegistre(tyEspece *registre, int nbTotal)` qui permet d'afficher toutes les espèces du registre. Le paramètre `nbTotal` correspond au nombre total d'observations dans le relevé. L'affichage aura le format suivant (pour le relevé exemple) :

Registre des especes observees (10 observations) :

```
espece 1 : 3 observations (30.00%)
espece 3 : 4 observations (40.00%)
espece 5 : 2 observations (20.00%)
espece 7 : 1 observations (10.00%)
```

Nombre d'especes distinctes : 4

Vous devrez appeler la fonction `NbEspeces` pour obtenir le nombre d'espèces distinctes.

Question 9 (4 points)

La fonction permettant de construire le registre à partir d'un relevé vous est donnée ci-dessous. Cette fonction retourne le registre construit.

```
1  tyEspece *ConstruireRegistre(int *releve, int n) {
2      tyEspece *registre = NULL;
3      int i;
4
5      for(i = 0; i < n; i++) {
6          if(EstValide(releve[i]) == 1) {
7              registre = InsérerTrie(registre, releve[i]);
8          }
9      }
10     return registre;
11 }
```

On considère le relevé `3 1 5` de longueur 3. Ce relevé a été alloué dans une autre fonction et se trouve dans le tas. Représenter la mémoire (pile et tas) à la fin du premier tour de la boucle `for` (c'est-à-dire après l'insertion du code 3). Le pointeur `releve` reçu en argument pointe sur le début du tableau dans le tas.

Question 10 (4 points)

Représenter la mémoire (pile et tas) à la fin du troisième et dernier tour de la boucle `for` (c'est-à-dire après l'insertion du code 5 dans le registre qui contient déjà les espèces 1 et 3). On rappelle que le registre est trié par code d'espèce croissant.

Question 11 (6 points)

Les espèces rarement observées ne sont pas significatives pour l'étude. Écrire la fonction de prototype `tyEspece *FiltrerRares(tyEspece *registre, int seuilMin)` qui permet de supprimer du registre toutes les espèces dont le nombre d'observations (`nbObs`) est strictement inférieur à `seuilMin`. Attention à bien libérer la mémoire des éléments supprimés. La fonction retourne un pointeur vers la tête de liste après filtrage.

Question 12 (4 points)

Écrire la fonction `main` du programme dans laquelle :

- Vous générerez aléatoirement un relevé de 1000 observations.
- Vous construirez le registre des espèces à partir de ce relevé.
- Vous afficherez le registre complet.
- Vous filtrerez les espèces ayant moins de 50 observations, puis afficherez le registre filtré.
- Vous libérerez toute la mémoire allouée.